

Hướng dẫn chuyên sâu về Security và Transaction Management trong Spring Boot

Bảo mật (Security) và Quản lý giao dịch (Transaction Management) là hai trong số những khía cạnh quan trọng và phức tạp nhất khi xây dựng các ứng dụng doanh nghiệp. Spring Boot, cùng với Spring Framework, cung cấp các giải pháp mạnh mẽ, linh hoạt và toàn diện để giải quyết hai vấn đề này. Tài liệu này sẽ đi sâu vào tất cả các khía cạnh của Spring Security và Transaction Management, từ các khái niệm cơ bản đến các kỹ thuật nâng cao, giúp bạn xây dựng các ứng dụng an toàn, ổn định và đáng tin cậy.

Phần I: Spring Security - Xây dựng ứng dụng an toàn

Mục lục phần I

- [Giới thiệu về Spring Security](#) 1.1. [Spring Security là gì?](#) 1.2. [Tại sao cần Spring Security?](#) 1.3. [Các tính năng chính](#)
- [Kiến trúc và các khái niệm cốt lõi](#) 2.1. [Tổng quan về kiến trúc Servlet](#) 2.2. [Authentication \(Xác thực\)](#) 2.3. [Authorization \(Ủy quyền\)](#) 2.4. [Principal, GrantedAuthority, và Role](#) 2.5. [SecurityContextHolder, SecurityContext, và Authentication Object](#)
- [Cấu hình Spring Security cơ bản với Spring Boot](#) 3.1. [Spring Boot Auto-configuration cho Security](#) 3.2. [Tùy chỉnh Security Filter Chain](#) 3.3. [Cấu hình UserDetailsService](#) 3.4. [Cấu hình PasswordEncoder](#)
- [Xác thực \(Authentication\) chuyên sâu](#) 4.1. [Form-based Authentication](#) 4.2. [Basic Authentication](#) 4.3. [In-Memory Authentication](#) 4.4. [JDBC-based Authentication](#) 4.5. [LDAP Authentication](#) 4.6. [OAuth 2.0 và OpenID Connect 1.0](#)
- [Ủy quyền \(Authorization\) chuyên sâu](#) 5.1. [Ủy quyền dựa trên Role và Authority](#) 5.2. [Ủy quyền dựa trên biểu thức \(Expression-Based Authorization\)](#) 5.3. [Method-level Security với @PreAuthorize, @PostAuthorize, @Secured, @RolesAllowed](#) 5.4. [Access Control Lists \(ACLs\)](#)

6. [Bảo vệ chống lại các cuộc tấn công phổ biến](#) 6.1. [Cross-Site Request Forgery \(CSRF\)](#) 6.2. [Cross-Site Scripting \(XSS\)](#) 6.3. [Session Fixation](#) 6.4. [Clickjacking](#) 6.5. [Bảo mật Header với Spring Security](#)
 7. [Tích hợp Spring Security với REST APIs](#) 7.1. [Stateless Authentication](#) 7.2. [JSON Web Token \(JWT\)](#) 7.3. [Tích hợp JWT với Spring Security](#)
 8. [Kiểm thử \(Testing\) ứng dụng Spring Security](#) 8.1. [Sử dụng @WithMockUser và @WithUserDetails](#) 8.2. [Kiểm thử với MockMvc](#)
-

1. Giới thiệu về Spring Security

1.1. Spring Security là gì?

Spring Security là một framework mạnh mẽ và có khả năng tùy biến cao, cung cấp các giải pháp xác thực (authentication) và ủy quyền (authorization) cho các ứng dụng Java. Nó là một dự án con của Spring Framework và đã trở thành tiêu chuẩn de-facto để bảo mật các ứng dụng được xây dựng trên nền tảng Spring.

1.2. Tại sao cần Spring Security?

Trong bất kỳ ứng dụng nào, việc bảo mật là tối quan trọng. Chúng ta cần đảm bảo rằng:

- Chỉ những người dùng hợp lệ mới có thể truy cập vào hệ thống (xác thực).
- Người dùng chỉ có thể thực hiện các hành động mà họ được phép (ủy quyền).
- Dữ liệu nhạy cảm của người dùng được bảo vệ.
- Ứng dụng được bảo vệ khỏi các cuộc tấn công phổ biến.

Việc tự xây dựng một hệ thống bảo mật từ đầu là vô cùng phức tạp, tốn thời gian và dễ mắc lỗi. Spring Security cung cấp một nền tảng vững chắc, đã được kiểm chứng và được cộng đồng tin dùng, giúp các nhà phát triển dễ dàng tích hợp các tính năng bảo mật vào ứng dụng của mình.

1.3. Các tính năng chính

- **Xác thực và ủy quyền toàn diện:** Hỗ trợ nhiều cơ chế xác thực khác nhau (form-based, basic, LDAP, OAuth 2.0, v.v.) và các chiến lược ủy quyền linh hoạt.

- **Bảo vệ chống lại các cuộc tấn công phổ biến:** Tích hợp sẵn các cơ chế bảo vệ chống lại CSRF, XSS, Session Fixation, Clickjacking, và nhiều loại tấn công khác.
 - **Tích hợp dễ dàng với Spring:** Hoạt động liền mạch với các dự án Spring khác, đặc biệt là Spring Boot.
 - **Khả năng tùy biến cao:** Hầu hết mọi khía cạnh của Spring Security đều có thể được tùy chỉnh để phù hợp với yêu cầu cụ thể của ứng dụng.
 - **Hỗ trợ cả ứng dụng Servlet và Reactive:** Cung cấp các giải pháp bảo mật cho cả hai mô hình lập trình web phổ biến trong Spring.
-

2. Kiến trúc và các khái niệm cốt lõi

Để sử dụng Spring Security hiệu quả, việc hiểu rõ kiến trúc và các thành phần cốt lõi của nó là rất quan trọng.

2.1. Tổng quan về kiến trúc Servlet

Trong một ứng dụng web dựa trên Servlet, Spring Security hoạt động như một chuỗi các bộ lọc (filters). Mỗi request đến sẽ đi qua chuỗi bộ lọc này trước khi đến được controller. Mỗi bộ lọc có một nhiệm vụ cụ thể, ví dụ:

- `UsernamePasswordAuthenticationFilter` : Xử lý xác thực dựa trên form đăng nhập.
- `BasicAuthenticationFilter` : Xử lý Basic Authentication.
- `CsrfFilter` : Bảo vệ chống lại tấn công CSRF.
- `FilterSecurityInterceptor` : Thực hiện ủy quyền, quyết định xem một request có được phép truy cập tài nguyên hay không.

Chuỗi bộ lọc này được quản lý bởi `FilterChainProxy`, một bean đặc biệt của Spring Security.

2.2. Authentication (Xác thực)

Xác thực là quá trình xác minh danh tính của một người dùng hoặc hệ thống. Trong Spring Security, quá trình này thường bao gồm các bước sau:

1. Người dùng cung cấp thông tin xác thực (ví dụ: username và password).

2. Thông tin này được đóng gói thành một đối tượng `Authentication` (thường là `UsernamePasswordAuthenticationToken`).
3. Đối tượng `Authentication` được chuyển đến `AuthenticationManager`.
4. `AuthenticationManager` sẽ ủy thác cho một hoặc nhiều `AuthenticationProvider` để thực hiện việc xác thực.
5. `AuthenticationProvider` sẽ sử dụng `UserDetailsService` để tải thông tin người dùng từ một nguồn dữ liệu (ví dụ: database, LDAP) và so sánh với thông tin được cung cấp.
6. Nếu xác thực thành công, `AuthenticationManager` sẽ trả về một đối tượng `Authentication` đã được xác thực đầy đủ (bao gồm cả các quyền - authorities).

2.3. Authorization (Ủy quyền)

Ủy quyền là quá trình quyết định xem một người dùng đã được xác thực có quyền truy cập vào một tài nguyên cụ thể hay không. Spring Security thực hiện ủy quyền sau khi quá trình xác thực đã thành công.

Quá trình này thường được xử lý bởi `FilterSecurityInterceptor`, dựa trên các quy tắc được cấu hình. Các quy tắc này có thể dựa trên vai trò (roles), quyền (authorities), hoặc các biểu thức phức tạp hơn.

2.4. Principal, GrantedAuthority, và Role

- **Principal:** Đại diện cho thực thể đã được xác thực. Nó có thể là một đối tượng `UserDetails` của Spring Security, hoặc bất kỳ đối tượng nào khác đại diện cho người dùng.
- **GrantedAuthority:** Đại diện cho một quyền được cấp cho principal. Ví dụ: `READ_PRIVILEGE`, `WRITE_PRIVILEGE`.
- **Role:** Là một `GrantedAuthority` có quy ước đặt tên bắt đầu bằng `ROLE_`. Ví dụ: `ROLE_ADMIN`, `ROLE_USER`. Spring Security cung cấp các phương thức tiện ích để làm việc với roles.

2.5. SecurityContextHolder, SecurityContext, và Authentication Object

- **SecurityContextHolder:** Là nơi Spring Security lưu trữ thông tin về principal hiện tại. Nó sử dụng `ThreadLocal` để đảm bảo rằng `SecurityContext` luôn có sẵn cho luồng thực thi hiện tại.

- **SecurityContext** : Chứa đối tượng **Authentication** và có thể chứa các thông tin bảo mật khác liên quan đến request.
 - **Authentication** : Đại diện cho thông tin xác thực của người dùng. Nó chứa **principal**, **credentials** (ví dụ: password), và danh sách các **GrantedAuthority**.
-

3. Cấu hình Spring Security cơ bản với Spring Boot

Spring Boot giúp việc cấu hình Spring Security trở nên đơn giản hơn rất nhiều nhờ vào cơ chế auto-configuration. Tuy nhiên, bạn vẫn có thể tùy chỉnh để phù hợp với nhu cầu cụ thể của ứng dụng.

3.1. Spring Boot Auto-configuration cho Security

Khi bạn thêm `spring-boot-starter-security` vào `pom.xml`, Spring Boot sẽ tự động cấu hình một số thứ mặc định:

- **Tạo một người dùng mặc định**: Nếu không có cấu hình người dùng nào được định nghĩa, Spring Security sẽ tạo một người dùng `user` với mật khẩu ngẫu nhiên (được in ra console khi ứng dụng khởi động).
- **Bảo vệ tất cả các endpoint**: Mặc định, tất cả các request đến ứng dụng đều yêu cầu xác thực.
- **Cung cấp trang đăng nhập mặc định**: Nếu bạn sử dụng form-based authentication, Spring Security sẽ tự động tạo một trang đăng nhập.
- **Cấu hình PasswordEncoder** : Sử dụng `BCryptPasswordEncoder` làm mặc định.

Thêm Dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

3.2. Tùy chỉnh Security Filter Chain

Để tùy chỉnh hành vi bảo mật, bạn cần định nghĩa một lớp cấu hình kế thừa `WebSecurityConfigurerAdapter` (đối với Spring Security 5.x trở về trước) hoặc định nghĩa các `SecurityFilterChain` bean (đối với Spring Security 6+). Với Spring Boot 3 (Spring Security 6+), cách tiếp cận được khuyến nghị là sử dụng `SecurityFilterChain` bean.

Ví dụ cấu hình cơ bản với Spring Security 6+:

```

package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

import static org.springframework.security.config.Customizer.withDefaults;

@Configuration
@EnableWebSecurity // Kích hoạt Spring Security
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .authorizeHttpRequests(authorize -> authorize
                .requestMatchers("/public/**").permitAll() // Cho phép truy cập
                các URL bắt đầu bằng /public mà không cần xác thực
                .requestMatchers("/admin/**").hasRole("ADMIN") // Yêu cầu vai trò
                ADMIN cho các URL /admin
                .anyRequest().authenticated() // Tất cả các request khác đều
                yêu cầu xác thực
            )
            .formLogin(withDefaults()) // Kích hoạt form login mặc định
            .httpBasic(withDefaults()); // Kích hoạt HTTP Basic authentication
        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService(PasswordEncoder passwordEncoder)
    {
        UserDetails user = User.withUsername("user")
            .password(passwordEncoder.encode("password"))
            .roles("USER")
            .build();
        UserDetails admin = User.withUsername("admin")
            .password(passwordEncoder.encode("admin"))
            .roles("ADMIN", "USER")
            .build();
        return new InMemoryUserDetailsManager(user, admin);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

Trong ví dụ trên:

- `@Configuration` và `@EnableWebSecurity` đánh dấu lớp này là lớp cấu hình Spring Security.
- `securityFilterChain` bean định nghĩa các quy tắc bảo mật:
 - `requestMatchers("/public/**").permitAll()` : Cho phép truy cập công khai vào các đường dẫn `/public`.
 - `requestMatchers("/admin/**").hasRole("ADMIN")` : Chỉ cho phép người dùng có vai trò `ADMIN` truy cập các đường dẫn `/admin`.
 - `anyRequest().authenticated()` : Tất cả các đường dẫn khác đều yêu cầu xác thực.
 - `formLogin(withDefaults())` : Kích hoạt trang đăng nhập mặc định của Spring Security.
 - `httpBasic(withDefaults())` : Kích hoạt HTTP Basic authentication.

3.3. Cấu hình `UserDetailsService`

`UserDetailsService` là một interface cốt lõi trong Spring Security, được sử dụng để tải thông tin người dùng (username, password, roles) từ một nguồn dữ liệu (ví dụ: database, LDAP, in-memory). Khi một người dùng cố gắng đăng nhập, Spring Security sẽ gọi phương thức `loadUserByUsername(String username)` của `UserDetailsService` để lấy thông tin người dùng.

Ví dụ về `UserDetailsService` tùy chỉnh (tải từ database):

Giả sử bạn có một `UserRepository` để truy cập thông tin người dùng từ database.


```

package com.example.demo.service;

import com.example.demo.model.UserEntity; // Gia' su' bạn có UserEntity
import com.example.demo.repository.UserRepository; // Gia' su' bạn có
UserRepository
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import static org.springframework.security.core.userdetails.User.withUsername;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        UserEntity userEntity = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found
with username: " + username));

        return withUsername(userEntity.getUsername())
            .password(userEntity.getPassword())
            .roles(userEntity.getRoles().toArray(new String[0])) // Gia' su'
roles là List<String>
            .build();
    }
}

```

Sau đó, bạn cần đăng ký `CustomUserDetailsService` này trong cấu hình bảo mật của mình. Spring Boot sẽ tự động phát hiện và sử dụng `UserDetailsService` bean mà bạn định nghĩa.

3.4. Cấu hình PasswordEncoder

`PasswordEncoder` là một interface được sử dụng để mã hóa (encode) mật khẩu. **Không bao giờ lưu trữ mật khẩu dưới dạng văn bản thuần túy (plain text)**. Spring Security khuyến nghị sử dụng `BCryptPasswordEncoder` vì tính bảo mật cao của nó.

Bạn cần định nghĩa một `PasswordEncoder` bean trong lớp cấu hình bảo mật của mình:

```

package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class PasswordEncoderConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

Khi lưu mật khẩu vào database, bạn sẽ mã hóa chúng bằng `passwordEncoder.encode("your_password")`. Khi xác thực, Spring Security sẽ tự động sử dụng `PasswordEncoder` để so sánh mật khẩu được cung cấp với mật khẩu đã mã hóa trong database.

4. Xác thực (Authentication) chuyên sâu

Spring Security cung cấp nhiều cơ chế xác thực khác nhau để phù hợp với các loại ứng dụng và yêu cầu bảo mật khác nhau.

4.1. Form-based Authentication

Đây là cơ chế xác thực phổ biến nhất cho các ứng dụng web truyền thống, nơi người dùng điền thông tin đăng nhập vào một form HTML. Spring Security cung cấp hỗ trợ mạnh mẽ cho form-based authentication, bao gồm cả trang đăng nhập mặc định và khả năng tùy chỉnh.

Cấu hình:

```

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
{
    http
        .authorizeHttpRequests(authorize -> authorize
            .anyRequest().authenticated()
        )
        .formLogin(form -> form
            .loginPage("/login") // Chỉ định URL của trang đăng nhập tùy chỉnh
            .permitAll() // Cho phép tất cả mọi người truy cập trang đăng nhập
        );
    return http.build();
}

```

- `loginPage("/login")` : Chỉ định URL mà Spring Security sẽ chuyển hướng đến khi yêu cầu xác thực. Nếu không được cấu hình, Spring Security sẽ sử dụng trang đăng nhập mặc định.
- `permitAll()` : Đảm bảo rằng trang đăng nhập có thể được truy cập bởi tất cả mọi người, kể cả những người chưa được xác thực.

Tạo trang đăng nhập tùy chỉnh (login.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Login</title>
</head>
<body>
  <h2>Login</h2>
  <form action="/login" method="post">
    <div>
      <label for="username">Username:</label>
      <input type="text" id="username" name="username"/>
    </div>
    <div>
      <label for="password">Password:</label>
      <input type="password" id="password" name="password"/>
    </div>
    <div>
      <input type="submit" value="Login"/>
    </div>
  </form>
</body>
</html>
```

Spring Security sẽ tự động xử lý request POST đến `/login` với các tham số `username` và `password`.

4.2. Basic Authentication

HTTP Basic Authentication là một cơ chế xác thực đơn giản, nơi thông tin đăng nhập (username và password) được gửi trong header `Authorization` của mỗi request, được mã hóa Base64. Nó thường được sử dụng cho các API nội bộ hoặc các ứng dụng client-side đơn giản.

Cấu hình:

```

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
{
    http
        .authorizeHttpRequests(authorize -> authorize
            .anyRequest().authenticated()
        )
        .httpBasic(withDefaults()); // Kích hoạt HTTP Basic authentication
    return http.build();
}

```

Khi kích hoạt `httpBasic(withDefaults())`, trình duyệt sẽ hiển thị một hộp thoại đăng nhập khi truy cập vào một tài nguyên được bảo vệ.

4.3. In-Memory Authentication

Đây là cách xác thực đơn giản nhất, nơi thông tin người dùng được lưu trữ trực tiếp trong bộ nhớ ứng dụng. Thường được sử dụng cho mục đích phát triển hoặc kiểm thử.

Cấu hình:

```

@Bean
public UserDetailsService userDetailsService(PasswordEncoder passwordEncoder) {
    UserDetails user = User.withUsername("user")
        .password(passwordEncoder.encode("password"))
        .roles("USER")
        .build();
    UserDetails admin = User.withUsername("admin")
        .password(passwordEncoder.encode("admin"))
        .roles("ADMIN", "USER")
        .build();
    return new InMemoryUserDetailsManager(user, admin);
}

```

Bạn đã thấy ví dụ này ở phần 3.2. `InMemoryUserDetailsManager` là implementation của `UserDetailsService` để quản lý người dùng trong bộ nhớ.

4.4. JDBC-based Authentication

Xác thực dựa trên JDBC cho phép bạn lưu trữ thông tin người dùng trong một cơ sở dữ liệu quan hệ. Spring Security cung cấp các công cụ để dễ dàng tích hợp với database của bạn.

Cấu hình:

```

package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.JdbcUserDetailsManager;

import javax.sql.DataSource;

@Configuration
public class JdbcSecurityConfig {

    @Bean
    public UserDetailsService jdbcUserDetailsService(DataSource dataSource,
        PasswordEncoder passwordEncoder) {
        JdbcUserDetailsManager userDetailsManager = new
        JdbcUserDetailsManager(dataSource);
        // Bạn có thể tùy chỉnh các câu truy vấn SQL nếu cấu trúc bảng của
        // bạn khác mặc định
        // userDetailsManager.setUsersByUsernameQuery("select username, password,
        // enabled from users where username = ?");
        // userDetailsManager.setAuthoritiesByUsernameQuery("select username,
        // authority from authorities where username = ?");

        // Ví dụ thêm người dùng nếu chưa có (chỉ cho mục đích demo)
        if (!userDetailsManager.userExists("jdbcuser")) {
            userDetailsManager.createUser(
                org.springframework.security.core.userdetails.User.withUsername("jdbcuser")
                    .password(passwordEncoder.encode("jdbcpassword"))
                    .roles("USER")
                    .build()
            );
        }
        return userDetailsManager;
    }
}

```

Bạn cần có các bảng `users` và `authorities` (hoặc các bảng tùy chỉnh) trong database của mình với cấu trúc phù hợp với `JdbcUserDetailsManager`.

4.5. LDAP Authentication

LDAP (Lightweight Directory Access Protocol) là một giao thức để truy cập và duy trì các dịch vụ thông tin thư mục phân tán. Spring Security cung cấp hỗ trợ để xác thực người dùng dựa trên máy chủ LDAP.

Thêm Dependency:

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-ldap</artifactId>
</dependency>

```

Cấu hình:

```
package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

import static org.springframework.security.config.Customizer.withDefaults;

@Configuration
public class LdapSecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .authorizeHttpRequests(authorize -> authorize
                .anyRequest().authenticated()
            )
            .formLogin(withDefaults())
            .ldap(withDefaults()); // Kích hoạt LDAP authentication
        return http.build();
    }

    // Cấu hình LDAP server trong application.properties
    // spring.ldap.urls=ldap://localhost:8389
    // spring.ldap.base=dc=springframework,dc=org
    // spring.ldap.username=uid=admin,ou=system
    // spring.ldap.password=secret

    // Cấu hình UserDetailsService cho LDAP
    // @Bean
    // public UserDetailsService userDetailsService() {
    //     return new LdapUserDetailsService(new
DefaultLdapAuthoritiesPopulator(...));
    // }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); // Hoặc NoOpPasswordEncoder nếu LDAP
không mã hóa
    }
}
```

4.6. OAuth 2.0 và OpenID Connect 1.0

OAuth 2.0 là một framework ủy quyền, cho phép ứng dụng của bên thứ ba truy cập vào tài nguyên được bảo vệ của người dùng trên một máy chủ tài nguyên (resource server). OpenID Connect (OIDC) là một lớp xác thực được xây dựng trên OAuth 2.0, cho phép client xác minh danh tính của người dùng dựa trên xác thực được thực hiện bởi Authorization Server.

Spring Security cung cấp hỗ trợ mạnh mẽ cho cả OAuth 2.0 (client và resource server) và OpenID Connect.

Thêm Dependency (ví dụ cho OAuth2 Client):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

Cấu hình (ví dụ cho Google OAuth2):

application.properties :

```
spring.security.oauth2.client.registration.google.client-id=YOUR_GOOGLE_CLIENT_ID
spring.security.oauth2.client.registration.google.client-secret=YOUR_GOOGLE_CLIENT_SECRET
spring.security.oauth2.client.registration.google.scope=openid,profile,email
```

SecurityConfig.java :

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
{
    http
        .authorizeHttpRequests(authorize -> authorize
            .anyRequest().authenticated()
        )
        .oauth2Login(withDefaults()); // Kích hoạt OAuth2 Login
    return http.build();
}
```

Khi người dùng truy cập vào một tài nguyên được bảo vệ, họ sẽ được chuyển hướng đến trang đăng nhập của Google để xác thực. Sau khi xác thực thành công, Google sẽ chuyển hướng người dùng trở lại ứng dụng của bạn với một mã ủy quyền (authorization code), và Spring Security sẽ xử lý phần còn lại.

5. Ủy quyền (Authorization) chuyên sâu

Sau khi người dùng đã được xác thực, bước tiếp theo là ủy quyền: quyết định xem người dùng đó có quyền truy cập vào tài nguyên hoặc thực hiện hành động cụ thể hay không.

5.1. Ủy quyền dựa trên Role và Authority

Đây là hình thức ủy quyền phổ biến nhất, nơi quyền truy cập được cấp dựa trên các vai trò (roles) hoặc quyền (authorities) mà người dùng có.

Cấu hình trong `SecurityFilterChain` :

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
{
    http
        .authorizeHttpRequests(authorize -> authorize
            .requestMatchers("/admin/**").hasRole("ADMIN") // Chỉ ADMIN mới truy cập /admin
            .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN") // USER hoặc ADMIN truy cập /user
            .requestMatchers("/api/products").hasAuthority("READ_PRODUCT") // Yêu cầu quyền READ_PRODUCT
            .anyRequest().authenticated()
        )
        // ...
    return http.build();
}
```

- `hasRole("ROLE_NAME")` : Kiểm tra xem người dùng có vai trò cụ thể hay không. Spring Security tự động thêm tiền tố `ROLE_` vào tên vai trò.
- `hasAnyRole("ROLE1", "ROLE2")` : Kiểm tra xem người dùng có bất kỳ vai trò nào trong danh sách hay không.
- `hasAuthority("AUTHORITY_NAME")` : Kiểm tra xem người dùng có quyền cụ thể hay không. Không thêm tiền tố tự động.

5.2. Ủy quyền dựa trên biểu thức (Expression-Based Authorization)

Spring Security cung cấp một ngôn ngữ biểu thức mạnh mẽ (Spring Expression Language - SpEL) để định nghĩa các quy tắc ủy quyền phức tạp hơn. Các biểu thức này có thể được sử dụng trong `authorizeHttpRequests` và trên các phương thức.

Ví dụ trong `SecurityFilterChain` :


```

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
{
    http
        .authorizeHttpRequests(authorize -> authorize
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .requestMatchers("/users/{username}").access(new
WebExpressionAuthorizationManager("hasRole('ADMIN') or #username ==
authentication.name"))
            .anyRequest().authenticated()
        )
        // ...
    return http.build();
}

```

Trong ví dụ trên, chỉ người dùng ADMIN hoặc người dùng có username trùng với {username} trong URL mới có thể truy cập /users/{username}.

5.3. Method-level Security với @PreAuthorize, @PostAuthorize, @Secured, @RolesAllowed

Ngoài việc bảo vệ các URL, bạn có thể bảo vệ các phương thức cụ thể trong service hoặc controller bằng cách sử dụng các annotation cấp phương thức. Để kích hoạt tính năng này, bạn cần thêm @EnableMethodSecurity vào lớp cấu hình bảo mật của mình.

```

package com.example.demo.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;

@Configuration
@EnableMethodSecurity(prePostEnabled = true, securedEnabled = true, jsr250Enabled = true)
public class MethodSecurityConfig {
    // Không cần thêm gì ở đây, chỉ cần annotation để kích hoạt
}

```

- **@PreAuthorize("expression")** : Kiểm tra quyền trước khi phương thức được thực thi. Đây là annotation mạnh mẽ nhất, cho phép sử dụng SpEL.

```

```java @PreAuthorize("hasRole('ADMIN')") public Product createProduct(Product product) { ... }

```

```

@PreAuthorize("hasRole('ADMIN') or #username == authentication.name") public UserProfile getUserProfile(String username) { ... } ```

```

- **@PostAuthorize("expression")** : Kiểm tra quyền sau khi phương thức được thực thi, thường được sử dụng để kiểm tra quyền truy cập vào đối tượng được trả về.

```
java @PostAuthorize("returnObject.owner == authentication.name") public
Document getDocumentById(Long id) { ... }
```

- **@Secured({"ROLE\_ADMIN", "ROLE\_USER"})**: Kiểm tra quyền dựa trên vai trò. Đơn giản hơn `@PreAuthorize` nhưng ít linh hoạt hơn.

```
java @Secured("ROLE_ADMIN") public void deleteUser(Long userId) { ... }
```

- **@RolesAllowed({"ADMIN", "USER"}) (JSR-250)**: Tương tự như `@Secured`, nhưng là một tiêu chuẩn JSR-250. Yêu cầu `jsr250Enabled = true` trong `@EnableMethodSecurity`.

```
java @RolesAllowed("ADMIN") public void updateSettings(Settings
settings) { ... }
```

## 5.4. Access Control Lists (ACLs)

Trong một số trường hợp, ủy quyền cần phải chi tiết hơn vai trò hoặc quyền chung. ACLs cho phép bạn định nghĩa quyền truy cập ở cấp độ đối tượng (ví dụ: người dùng A có thể chỉnh sửa tài liệu X, nhưng không thể chỉnh sửa tài liệu Y). Spring Security cung cấp một module ACL riêng biệt.

ACLs phức tạp hơn để cấu hình và thường chỉ được sử dụng trong các ứng dụng có yêu cầu bảo mật rất chi tiết ở cấp độ đối tượng.

---

## 6. Bảo vệ chống lại các cuộc tấn công phổ biến

Spring Security không chỉ cung cấp xác thực và ủy quyền mà còn tích hợp sẵn các biện pháp bảo vệ chống lại nhiều loại tấn công web phổ biến.

### 6.1. Cross-Site Request Forgery (CSRF)

CSRF là một cuộc tấn công buộc người dùng cuối thực hiện các hành động không mong muốn trên một ứng dụng web mà họ đã đăng nhập. Spring Security cung cấp bảo vệ CSRF mặc định bằng cách yêu cầu một CSRF token trong mỗi request POST, PUT, DELETE.

#### Cách hoạt động:

- Khi người dùng truy cập trang, Spring Security sẽ tạo một CSRF token và đặt nó vào session.

- Token này được nhúng vào form HTML hoặc header của request.
- Khi request được gửi, Spring Security sẽ kiểm tra xem token trong request có khớp với token trong session hay không.

### Cấu hình:

Bảo vệ CSRF được bật mặc định. Bạn có thể tắt nó nếu cần (ví dụ: cho các API stateless không sử dụng session), nhưng không khuyến nghị:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
{
 http
 .csrf(csrf -> csrf.disable()) // Tắt bảo vệ CSRF (KHÔNG KHUYẾN NGHỊ cho
 // ...
 // ứng dụng web có trạng thái)
 .return http.build();
}
```

Đối với các ứng dụng RESTful API sử dụng JWT hoặc OAuth2 (stateless), bạn thường sẽ tắt CSRF vì nó không cần thiết và có thể gây phức tạp.

## 6.2. Cross-Site Scripting (XSS)

XSS là một loại tấn công tiêm mã độc vào trang web. Mặc dù Spring Security không trực tiếp ngăn chặn XSS (đây là trách nhiệm của việc mã hóa đầu ra và làm sạch đầu vào), nhưng nó cung cấp các header bảo mật có thể giúp giảm thiểu rủi ro.

### Biện pháp phòng ngừa:

- **Mã hóa đầu ra (Output Encoding):** Luôn mã hóa bất kỳ dữ liệu nào được hiển thị trên trang web mà có nguồn gốc từ người dùng.
- **Làm sạch đầu vào (Input Sanitization):** Loại bỏ hoặc làm sạch các ký tự đặc biệt từ dữ liệu đầu vào của người dùng.
- **Content Security Policy (CSP):** Sử dụng CSP để hạn chế các nguồn tài nguyên mà trình duyệt có thể tải và thực thi.

## 6.3. Session Fixation

Session Fixation là một cuộc tấn công trong đó kẻ tấn công cố gắng buộc người dùng sử dụng một ID session mà kẻ tấn công đã biết. Spring Security bảo vệ chống lại điều này bằng cách tạo một session ID mới khi người dùng đăng nhập thành công.

## Cấu hình:

Bảo vệ Session Fixation được bật mặc định. Bạn có thể cấu hình chiến lược bảo vệ:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
{
 http
 .sessionManagement(session -> session
 .sessionFixation().migrateSession() // Mặc định: tạo session mới và di
 // chuyển thuộc tính
 // .sessionFixation().newSession() // Luôn tạo session mới
 // .sessionFixation().none() // Không bảo vệ (KHÔNG KHUYẾN NGHỊ)
)
 // ...
 return http.build();
}
```

## 6.4. Clickjacking

Clickjacking là một cuộc tấn công trong đó kẻ tấn công lừa người dùng nhấp vào một phần tử ẩn trên một trang web độc hại, nhưng thực chất lại đang nhấp vào một phần tử trên trang web hợp pháp. Spring Security bảo vệ chống lại Clickjacking bằng cách sử dụng `X-Frame-Options` header.

## Cấu hình:

Bảo vệ Clickjacking được bật mặc định. Bạn có thể tùy chỉnh `X-Frame-Options`:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
{
 http
 .headers(headers -> headers
 .frameOptions(frameOptions -> frameOptions
 .deny() // Không cho phép nhúng trang trong iframe
 // .sameOrigin() // Chỉ cho phép nhúng từ cùng một origin
)
)
 // ...
 return http.build();
}
```

## 6.5. Bảo mật Header với Spring Security

Spring Security cung cấp một cách dễ dàng để thêm các header bảo mật vào phản hồi HTTP, giúp tăng cường bảo mật cho ứng dụng của bạn.

**Các header được hỗ trợ:**

- `X-Content-Type-Options` : Ngăn chặn trình duyệt

phát hiện kiểu MIME (MIME sniffing). - `X-XSS-Protection` : Kích hoạt bộ lọc XSS tích hợp trong trình duyệt. - `Strict-Transport-Security` (HSTS) : Buộc trình duyệt chỉ giao tiếp với máy chủ qua HTTPS. - `Content-Security-Policy` (CSP) : Ngăn chặn các cuộc tấn công XSS và tiêm dữ liệu khác bằng cách chỉ định các nguồn nội dung được phép. - `Referrer-Policy` : Kiểm soát thông tin referrer được gửi trong các request.

### Cấu hình:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
{
 http
 .headers(headers -> headers
 .xssProtection(withDefaults())
 .contentSecurityPolicy(csp -> csp.policyDirectives("script-src
'self'"))
 .frameOptions(frameOptions -> frameOptions.deny())
 .httpStrictTransportSecurity(hsts ->
hsts.includeSubDomains(true).maxAgeInSeconds(31536000))
 .referrerPolicy(referrer ->
referrer.policy(ReferrerPolicy.NO_REFERRER))
 .permissionsPolicy(permissions -> permissions.policy("geolocation=
(self)"))
)
 // ...
 return http.build();
}
```

## 7. Tích hợp Spring Security với REST APIs

Khi xây dựng các RESTful API, việc tích hợp bảo mật có một số khác biệt so với các ứng dụng web truyền thống sử dụng session. Các API thường là stateless, và JWT (JSON Web Token) là một giải pháp phổ biến.

### 7.1. Stateless Authentication

Trong các ứng dụng RESTful, mỗi request từ client đến server thường không phụ thuộc vào trạng thái session trước đó. Điều này có nghĩa là server không lưu trữ thông tin session của client. Để đạt được điều này, bạn cần cấu hình Spring Security để không tạo session.

### Cấu hình:

```

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception
{
 http
 .csrf(csrf -> csrf.disable()) // Tắt CSRF cho API stateless
 .sessionManagement(session ->
 session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // Không tạo session
 .authorizeHttpRequests(authorize -> authorize
 .anyRequest().authenticated()
)
 // ...
 return http.build();
}

```

## 7.2. JSON Web Token (JWT)

JWT là một tiêu chuẩn mở (RFC 7519) để tạo các token truy cập an toàn giữa hai bên. JWT là compact, URL-safe và có thể được sử dụng để truyền tải thông tin đã được xác minh giữa các bên.

Một JWT bao gồm ba phần, được phân tách bằng dấu chấm ( . ):

- **Header:** Chứa loại token (JWT) và thuật toán mã hóa (ví dụ: HMAC SHA256 hoặc RSA).
- **Payload:** Chứa các claims (tuyên bố) về thực thể (thường là người dùng) và các dữ liệu bổ sung. Các claims có thể là:
  - **Registered claims:** Các claims được định nghĩa trước (ví dụ: `iss` (issuer), `exp` (expiration time), `sub` (subject)).
  - **Public claims:** Các claims được định nghĩa bởi những người sử dụng JWT, nhưng phải được đăng ký trong IANA JSON Web Token Registry hoặc là một URI có chứa không gian tên và phạm vi.
  - **Private claims:** Các claims tùy chỉnh được tạo ra để chia sẻ thông tin giữa các bên đồng ý sử dụng chúng.
- **Signature:** Được tạo bằng cách mã hóa header, payload và một secret key. Chữ ký được sử dụng để xác minh rằng token không bị thay đổi trên đường truyền.

**Cấu trúc JWT:** `base64UrlEncode(header) + "." + base64UrlEncode(payload) + "." + signature`

## 7.3. Tích hợp JWT với Spring Security

Để tích hợp JWT với Spring Security, bạn cần thực hiện các bước sau:

1. **Thêm Dependency:** Thêm thư viện JWT (ví dụ: `jjwt` ).
2. **Tạo JWT Utility:** Một lớp để tạo và xác thực JWT.
3. **Tạo JWT Authentication Filter:** Một filter để chặn các request, trích xuất JWT và xác thực người dùng.
4. **Cấu hình Spring Security:** Thêm JWT filter vào chuỗi filter của Spring Security.

#### Thêm Dependency:

```
<dependency>
 <groupId>io.jsonwebtoken</groupId>
 <artifactId>jjwt-api</artifactId>
 <version>0.11.5</version>
</dependency>
<dependency>
 <groupId>io.jsonwebtoken</groupId>
 <artifactId>jjwt-impl</artifactId>
 <version>0.11.5</version>
 <scope>runtime</scope>
</dependency>
<dependency>
 <groupId>io.jsonwebtoken</groupId>
 <artifactId>jjwt-jackson</artifactId>
 <version>0.11.5</version>
 <scope>runtime</scope>
</dependency>
```

#### Ví dụ JWT Utility ( `JwtUtil.java` ):

```

package com.example.demo.security.jwt;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;

import java.security.Key;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

@Component
public class JwtUtil {

 @Value("${jwt.secret}")
 private String SECRET_KEY;

 @Value("${jwt.expiration}")
 private long EXPIRATION_TIME;

 public String extractUsername(String token) {
 return extractClaim(token, Claims::getSubject);
 }

 public Date extractExpiration(String token) {
 return extractClaim(token, Claims::getExpiration);
 }

 public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
 final Claims claims = extractAllClaims(token);
 return claimsResolver.apply(claims);
 }

 private Claims extractAllClaims(String token) {
 return
Jwts.parserBuilder().setSigningKey(getSigningKey()).build().parseClaimsJws(token).ge
 }

 private Boolean isTokenExpired(String token) {
 return extractExpiration(token).before(new Date());
 }

 public String generateToken(UserDetails userDetails) {
 Map<String, Object> claims = new HashMap<>();
 return createToken(claims, userDetails.getUsername());
 }

 private String createToken(Map<String, Object> claims, String subject) {
 return Jwts.builder().setClaims(claims).setSubject(subject)
 .setIssuedAt(new Date(System.currentTimeMillis()))
 .setExpiration(new Date(System.currentTimeMillis() +
EXPIRATION_TIME))
 .signWith(getSigningKey(), SignatureAlgorithm.HS256).compact();
 }

 public Boolean validateToken(String token, UserDetails userDetails) {
 final String username = extractUsername(token);

```



```
 return (username.equals(userDetails.getUsername()) &&
!isTokenExpired(token));
 }

 private Key getSigningKey() {
 byte[] keyBytes = Decoders.BASE64.decode(SECRET_KEY);
 return Keys.hmacShaKeyFor(keyBytes);
 }
}
```

**Ví dụ JWT Authentication Filter ( JwtRequestFilter.java ):**

```

package com.example.demo.security.jwt;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class JwtRequestFilter extends OncePerRequestFilter {

 private final UserDetailsService userDetailsService;
 private final JwtUtil jwtUtil;

 public JwtRequestFilter(UserDetailsService userDetailsService, JwtUtil
jwtUtil) {
 this.userDetailsService = userDetailsService;
 this.jwtUtil = jwtUtil;
 }

 @Override
 protected void doFilterInternal(HttpServletRequest request,
HttpServletResponse response, FilterChain chain)
 throws ServletException, IOException {

 final String authorizationHeader = request.getHeader("Authorization");

 String username = null;
 String jwt = null;

 if (authorizationHeader != null && authorizationHeader.startsWith("Bearer
")) {
 jwt = authorizationHeader.substring(7);
 username = jwtUtil.extractUsername(jwt);
 }

 if (username != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {

 UserDetails userDetails =
this.userDetailsService.loadUserByUsername(username);

 if (jwtUtil.validateToken(jwt, userDetails)) {

 UsernamePasswordAuthenticationToken
usernamePasswordAuthenticationToken = new UsernamePasswordAuthenticationToken(
 userDetails, null, userDetails.getAuthorities());
 usernamePasswordAuthenticationToken
 .setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));
 SecurityContextHolder.getContext().setAuthentication(usernamePasswordAuthenticationT
 }
 }
 }
}

```

```
 }
 chain.doFilter(request, response);
 }
}
```

**Cấu hình Spring Security để sử dụng JWT Filter:**

```

package com.example.demo.config;

import com.example.demo.security.jwt.JwtRequestFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
public class JwtSecurityConfig {

 private final JwtRequestFilter jwtRequestFilter;

 public JwtSecurityConfig(JwtRequestFilter jwtRequestFilter) {
 this.jwtRequestFilter = jwtRequestFilter;
 }

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
 http
 .csrf(csrf -> csrf.disable())
 .authorizeHttpRequests(authorize -> authorize
 .requestMatchers("/authenticate").permitAll() // Cho phép endpoint
 xác thực công khai
 .anyRequest().authenticated() // Các request khác yêu cầu xác
 thực
)
 .sessionManagement(session ->
 session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)); // Không tạo
 session

 http.addFilterBefore(jwtRequestFilter,
 UsernamePasswordAuthenticationFilter.class);

 return http.build();
 }

 @Bean
 public AuthenticationManager authenticationManager(AuthenticationConfiguration
 authenticationConfiguration) throws Exception {
 return authenticationConfiguration.getAuthenticationManager();
 }

 @Bean
 public PasswordEncoder passwordEncoder() {
 return new BCryptPasswordEncoder();
 }
}

```

Với cấu hình này, khi client gửi request đến `/authenticate` với username/password, server sẽ trả về một JWT. Client sau đó sẽ gửi JWT này trong header `Authorization: Bearer <token>` cho các request tiếp theo. `JwtRequestFilter` sẽ chặn các request này, xác thực JWT và thiết lập `SecurityContext`.

---

## 8. Kiểm thử (Testing) ứng dụng Spring Security

---

Kiểm thử các khía cạnh bảo mật của ứng dụng là rất quan trọng để đảm bảo rằng các quy tắc xác thực và ủy quyền hoạt động đúng như mong đợi. Spring Security cung cấp các tiện ích để giúp việc kiểm thử dễ dàng hơn.

### 8.1. Sử dụng `@WithMockUser` và `@WithUserDetails`

- `@WithMockUser`: Annotation này cho phép bạn chạy một test method với một người dùng giả lập (mock user) được tạo nhanh chóng. Bạn có thể chỉ định username, password và roles.

```
```java package com.example.demo.controller;
```

```
import org.junit.jupiter.api.Test; import
org.springframework.beans.factory.annotation.Autowired; import
org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest; import
org.springframework.security.test.context.support.WithMockUser; import
org.springframework.test.web.servlet.MockMvc;
```

```
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get; import
static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
```

```
@WebMvcTest(ProductController.class) public class ProductControllerSecurityTest {
```

```

@Autowired
private MockMvc mockMvc;

@Test
@WithMockUser(username = "user", roles = {"USER"})
void givenUserRole_whenAccessUserEndpoint_thenOk() throws Exception {
    mockMvc.perform(get("/user/dashboard"))
        .andExpect(status().isOk());
}

@Test
@WithMockUser(username = "admin", roles = {"ADMIN"})
void givenAdminRole_whenAccessAdminEndpoint_thenOk() throws Exception {
    mockMvc.perform(get("/admin/settings"))
        .andExpect(status().isOk());
}

@Test
@WithMockUser(username = "user", roles = {"USER"})
void givenUserRole_whenAccessAdminEndpoint_thenForbidden() throws Exception {
    mockMvc.perform(get("/admin/settings"))
        .andExpect(status().isForbidden());
}
}

```

- **@WithUserDetails**: Nếu bạn cần một `UserDetails` object phức tạp hơn (ví dụ: được tải từ `UserDetailsService`), bạn có thể sử dụng `@WithUserDetails`. Annotation này yêu cầu một bean `UserDetailsService` trong context test.

```

java // Giả sử bạn có một UserDetailsService bean được cấu hình trong
test context @Test @WithUserDetails("testUser") // testUser là username
của một UserDetails trong UserDetailsService void
givenSpecificUser_whenAccessProtectedResource_thenOk() throws Exception
{
    mockMvc.perform(get("/protected/resource"))
        .andExpect(status().isOk()); }

```

8.2. Kiểm thử với MockMvc

Bạn có thể sử dụng `MockMvc` để gửi các request giả lập và kiểm tra phản hồi HTTP, bao gồm cả status code và nội dung. Để kiểm thử các endpoint được bảo vệ, bạn cần sử dụng `SecurityMockMvcRequestPostProcessors`.

Thêm Dependency:

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

Ví dụ:

```
package com.example.demo.controller;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.web.servlet.MockMvc;

import static
org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcs
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@WebMvcTest(ProductController.class)
public class ProductControllerMockMvcSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void whenAccessProtectedEndpointWithoutAuth_thenUnauthorized() throws
Exception {
        mockMvc.perform(get("/protected/data"))
                .andExpect(status().isUnauthorized());
    }

    @Test
    void whenAccessProtectedEndpointWithUserAuth_thenOk() throws Exception {
mockMvc.perform(get("/protected/data").with(user("testUser").roles("USER")))
                .andExpect(status().isOk());
    }

    @Test
    void whenAccessAdminEndpointWithUserAuth_thenForbidden() throws Exception {
        mockMvc.perform(get("/admin/data").with(user("testUser").roles("USER")))
                .andExpect(status().isForbidden());
    }

    @Test
    void whenAccessAdminEndpointWithAdminAuth_thenOk() throws Exception {
        mockMvc.perform(get("/admin/data").with(user("adminUser").roles("ADMIN")))
                .andExpect(status().isOk());
    }
}
```

Phần II: Transaction Management - Đảm bảo tính nhất quán dữ liệu

Mục lục phần II

1. [Giới thiệu về Transaction Management](#) 1.1. [Transaction là gì?](#) 1.2. [Tính chất ACID của Transaction](#) 1.3. [Tại sao cần Transaction Management trong Spring?](#)
 2. [Các loại Transaction Management trong Spring](#) 2.1. [Programmatic Transaction Management](#) 2.2. [Declarative Transaction Management](#)
 3. [Declarative Transaction Management với @Transactional](#) 3.1. [Cách sử dụng @Transactional](#) 3.2. [Transaction Propagation \(Mức độ lan truyền giao dịch\)](#) 3.3. [Transaction Isolation \(Mức độ cô lập giao dịch\)](#) 3.4. [Rollback Rules \(Quy tắc Rollback\)](#) 3.5. [Timeout và Read-only Transactions](#)
 4. [Transaction Managers](#) 4.1. [PlatformTransactionManager](#) 4.2. [DataSourceTransactionManager](#) 4.3. [JpaTransactionManager](#) 4.4. [HibernateTransactionManager](#)
 5. [Tích hợp Transaction Management với Spring Boot](#) 5.1. [Auto-configuration cho Transaction Management](#) 5.2. [Cấu hình Transaction Manager tùy chỉnh](#)
 6. [Best Practices và Lời khuyên](#) 6.1. [Sử dụng @Transactional đúng cách](#) 6.2. [Xử lý ngoại lệ trong Transaction](#) 6.3. [Tránh các vấn đề về hiệu suất](#) 6.4. [Kiểm thử Transaction](#)
-

1. Giới thiệu về Transaction Management

1.1. Transaction là gì?

Trong ngữ cảnh của hệ thống quản lý cơ sở dữ liệu (DBMS) hoặc các hệ thống phân tán, một **transaction (giao dịch)** là một chuỗi các thao tác được thực hiện như một đơn vị công việc logic duy nhất. Điều này có nghĩa là tất cả các thao tác trong một transaction phải được hoàn thành thành công (commit) hoặc không có thao tác nào được hoàn thành (rollback). Transaction đảm bảo tính toàn vẹn và nhất quán của dữ liệu.

Ví dụ: Chuyển tiền từ tài khoản A sang tài khoản B. Transaction này bao gồm hai thao tác:

1. Trừ tiền từ tài khoản A.

2. Cộng tiền vào tài khoản B.

Nếu thao tác 1 thành công nhưng thao tác 2 thất bại (ví dụ: lỗi hệ thống), toàn bộ transaction phải được rollback để tiền không bị mất khỏi tài khoản A mà không được cộng vào tài khoản B.

1.2. Tính chất ACID của Transaction

Các transaction phải tuân thủ bốn tính chất quan trọng, thường được gọi là ACID:

- **Atomicity (Tính nguyên tử):** Một transaction là một đơn vị không thể chia nhỏ. Nó phải được thực hiện hoàn toàn hoặc không thực hiện gì cả. Không có trạng thái trung gian nào được nhìn thấy.
- **Consistency (Tính nhất quán):** Một transaction phải đưa cơ sở dữ liệu từ một trạng thái hợp lệ này sang một trạng thái hợp lệ khác. Nó phải tuân thủ tất cả các quy tắc và ràng buộc của cơ sở dữ liệu.
- **Isolation (Tính cô lập):** Các transaction đồng thời phải được thực hiện một cách độc lập với nhau. Kết quả của việc thực hiện nhiều transaction đồng thời phải giống như việc thực hiện chúng một cách tuần tự.
- **Durability (Tính bền vững):** Khi một transaction đã được commit, các thay đổi của nó phải được lưu trữ vĩnh viễn trong cơ sở dữ liệu và không bị mất ngay cả khi hệ thống gặp sự cố.

1.3. Tại sao cần Transaction Management trong Spring?

Trong các ứng dụng Java, đặc biệt là các ứng dụng doanh nghiệp, việc quản lý transaction một cách thủ công có thể rất phức tạp và dễ mắc lỗi. Spring Framework cung cấp một cơ chế quản lý transaction mạnh mẽ và linh hoạt, giúp đơn giản hóa việc này:

- **Tách biệt mối quan tâm:** Tách biệt logic quản lý transaction khỏi logic nghiệp vụ, giúp code sạch sẽ và dễ bảo trì.
- **Độc lập với công nghệ:** Spring Transaction Management hoạt động với nhiều công nghệ truy cập dữ liệu khác nhau (JDBC, JPA, Hibernate, JTA, JMS, v.v.), cho phép bạn thay đổi công nghệ mà không cần thay đổi logic transaction.
- **Declarative Transaction Management:** Cho phép bạn định nghĩa hành vi transaction bằng cách sử dụng annotation (`@Transactional`) hoặc XML, thay vì viết code quản lý transaction thủ công.

- **Tích hợp với Spring Boot:** Spring Boot tự động cấu hình Transaction Management khi phát hiện các dependency liên quan (ví dụ: Spring Data JPA).
-

2. Các loại Transaction Management trong Spring

Spring hỗ trợ hai loại quản lý transaction chính:

2.1. Programmatic Transaction Management

Quản lý transaction theo chương trình (programmatic) có nghĩa là bạn tự viết code để quản lý transaction (bắt đầu, commit, rollback). Spring cung cấp `PlatformTransactionManager` và `TransactionTemplate` để hỗ trợ việc này.

Sử dụng `PlatformTransactionManager` trực tiếp:

```

package com.example.demo.service;

import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.stereotype.Service;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.TransactionDefinition;
import org.springframework.transaction.TransactionStatus;
import org.springframework.transaction.support.DefaultTransactionDefinition;

import javax.sql.DataSource;

@Service
public class AccountService {

    private final PlatformTransactionManager transactionManager;
    private final DataSource dataSource; // Giả sử có DataSource để thực hiện
    các thao tác DB

    public AccountService(DataSource dataSource) {
        this.dataSource = dataSource;
        this.transactionManager = new DataSourceTransactionManager(dataSource);
    }

    public void transferMoney(Long fromAccountId, Long toAccountId, double amount)
    {
        DefaultTransactionDefinition def = new DefaultTransactionDefinition();
        def.setPropagationBehavior(TransactionDefinition.PROPGATION_REQUIRED);
        TransactionStatus status = transactionManager.getTransaction(def);

        try {
            // Logic trừ tiền từ fromAccountId
            // Logic cộng tiền vào toAccountId
            System.out.println("Transferring " + amount + " from " + fromAccountId
+ " to " + toAccountId);
            // Giả lập lỗi
            // if (true) throw new RuntimeException("Simulated error");

            transactionManager.commit(status);
            System.out.println("Transaction committed successfully.");
        } catch (Exception ex) {
            transactionManager.rollback(status);
            System.err.println("Transaction rolled back: " + ex.getMessage());
            throw ex;
        }
    }
}

```

Sử dụng TransactionTemplate (được khuyến nghị hơn):

TransactionTemplate là một cách tiếp cận đơn giản hơn để quản lý transaction theo chương trình, giúp giảm boilerplate code.

```

package com.example.demo.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.support.TransactionTemplate;

@Service
public class AccountServiceWithTemplate {

    private final TransactionTemplate transactionTemplate;

    public AccountServiceWithTemplate(PlatformTransactionManager
transactionManager) {
        this.transactionTemplate = new TransactionTemplate(transactionManager);
    }

    public void transferMoney(Long fromAccountId, Long toAccountId, double amount)
{
        transactionTemplate.execute(status -> {
            try {
                // Logic trừ tiền từ fromAccountId
                // Logic cộng tiền vào toAccountId
                System.out.println("Transferring " + amount + " from " +
fromAccountId + " to " + toAccountId);
                // Giả lập lỗi
                // if (true) throw new RuntimeException("Simulated error");
                return null; // Tra về null hoặc bất kỳ giá trị nào
            } catch (Exception ex) {
                status.setRollbackOnly(); // Đánh dấu transaction để rollback
                System.err.println("Error during transfer: " + ex.getMessage());
                throw ex; // Ném lại ngoại lệ để báo hiệu lỗi
            }
        });
    }
}

```

Ưu điểm:

- Kiểm soát transaction rất chi tiết.

Nhược điểm:

- Dễ dẫn đến boilerplate code và khó bảo trì nếu không được sử dụng cẩn thận.
- Không được khuyến nghị cho hầu hết các trường hợp.

2.2. Declarative Transaction Management

Quản lý transaction theo khai báo (declarative) là cách tiếp cận được khuyến nghị trong Spring. Nó cho phép bạn định nghĩa hành vi transaction bằng cách sử dụng annotation (`@Transactional`) hoặc cấu hình XML, mà không cần viết code quản lý transaction thủ công. Spring sẽ tự động tạo một proxy xung quanh các bean được đánh dấu để quản lý transaction.

Ưu điểm:

- **Đơn giản và sạch sẽ:** Giảm đáng kể boilerplate code.
- **Tách biệt mối quan tâm:** Logic nghiệp vụ không bị trộn lẫn với logic transaction.
- **Dễ bảo trì:** Dễ dàng thay đổi hành vi transaction mà không cần sửa đổi code nghiệp vụ.

Nhược điểm:

- Ít linh hoạt hơn so với programmatic transaction management trong các trường hợp rất phức tạp.

3. Declarative Transaction Management với

`@Transactional`

Annotation `@Transactional` là cách phổ biến nhất để quản lý transaction trong Spring Boot. Nó có thể được đặt ở cấp độ lớp hoặc cấp độ phương thức.

3.1. Cách sử dụng `@Transactional`

Khi đặt `@Transactional` trên một phương thức, Spring sẽ tạo một transaction mới trước khi phương thức đó được thực thi và commit transaction khi phương thức hoàn thành thành công. Nếu một `RuntimeException` (hoặc `Error`) được ném ra, transaction sẽ được rollback.

Khi đặt `@Transactional` trên một lớp, tất cả các phương thức public trong lớp đó sẽ được thực thi trong một transaction.

Ví dụ:

```

package com.example.demo.service;

import com.example.demo.model.Product;
import com.example.demo.repository.ProductRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    @Transactional
    public Product createProduct(Product product) {
        // Thao tác 1: Lưu sản phẩm
        Product savedProduct = productRepository.save(product);

        // Giả' lập một thao tác khác có thể' gây lỗi
        // if (true) throw new RuntimeException("Simulated error during product
creation");

        // Thao tác 2: Cập nhật một số' thông tin liên quan (ví dụ: tồn kho)
        // ...

        return savedProduct;
    }

    @Transactional
    public void updateProductStock(Long productId, int quantityChange) {
        Product product = productRepository.findById(productId)
            .orElseThrow(() -> new RuntimeException("Product not found"));
        product.setStock(product.getStock() + quantityChange);
        productRepository.save(product);
    }
}

```

Trong ví dụ `createProduct`, nếu bất kỳ lỗi nào xảy ra sau `productRepository.save(product)`, toàn bộ thao tác lưu sản phẩm sẽ được rollback.

3.2. Transaction Propagation (Mức độ lan truyền giao dịch)

Propagation behavior định nghĩa cách một transaction mới sẽ được bắt đầu hoặc cách một phương thức sẽ tham gia vào một transaction hiện có. Đây là một thuộc tính quan trọng của `@Transactional`.

Các mức độ propagation phổ biến:

- **REQUIRED (Mặc định)**: Nếu một transaction hiện có đang chạy, phương thức sẽ tham gia vào transaction đó. Nếu không, một transaction mới sẽ được tạo.

```
```java @Transactional(propagation = Propagation.REQUIRED) public void  
methodA() { // Tham gia vào transaction của caller hoặc tạo mới methodB(); }
```

```
@Transactional(propagation = Propagation.REQUIRED) public void methodB() { //
Tham gia vào transaction của methodA // ... } ```
```

- **SUPPORTS** : Nếu một transaction hiện có đang chạy, phương thức sẽ tham gia vào transaction đó. Nếu không, phương thức sẽ được thực thi mà không có transaction.
- **MANDATORY** : Phương thức phải được gọi trong một transaction hiện có. Nếu không có transaction nào đang chạy, một `TransactionRequiredException` sẽ được ném ra.
- **REQUIRES\_NEW** : Luôn tạo một transaction mới. Nếu một transaction hiện có đang chạy, nó sẽ bị tạm dừng cho đến khi transaction mới hoàn thành.

```
```java @Transactional(propagation = Propagation.REQUIRED) public void  
methodA() { // ... methodB(); // methodA's transaction bị tạm dừng // ... }
```

```
@Transactional(propagation = Propagation.REQUIRES_NEW) public void methodB()  
{ // Luôn tạo transaction mới // ... } ```
```

- **NOT_SUPPORTED** : Phương thức sẽ được thực thi mà không có transaction. Nếu một transaction hiện có đang chạy, nó sẽ bị tạm dừng.
- **NEVER** : Phương thức sẽ được thực thi mà không có transaction. Nếu một transaction hiện có đang chạy, một `IllegalTransactionStateException` sẽ được ném ra.
- **NESTED** : Nếu một transaction hiện có đang chạy, một savepoint sẽ được tạo. Nếu phương thức ném ngoại lệ, nó sẽ rollback về savepoint đó mà không ảnh hưởng đến transaction bên ngoài. Nếu không có transaction nào đang chạy, nó hoạt động như `REQUIRED`.

3.3. Transaction Isolation (Mức độ cô lập giao dịch)

Isolation level định nghĩa mức độ mà các thay đổi được thực hiện bởi một transaction có thể được nhìn thấy bởi các transaction đồng thời khác. Các mức độ cô lập khác nhau có thể ngăn chặn các vấn đề như:

- **Dirty Reads**: Một transaction đọc dữ liệu chưa được commit bởi một transaction khác.
- **Non-repeatable Reads**: Một transaction đọc cùng một hàng hai lần và nhận được các giá trị khác nhau do một transaction khác đã commit thay đổi giữa hai lần đọc.

- **Phantom Reads:** Một transaction đọc cùng một tập hợp các hàng hai lần và nhận được các hàng mới do một transaction khác đã commit thêm hàng mới giữa hai lần đọc.

Các mức độ cô lập (từ thấp đến cao): (Mức độ cao hơn cung cấp tính nhất quán tốt hơn nhưng có thể ảnh hưởng đến hiệu suất)

- **READ_UNCOMMITTED** : Cho phép dirty reads. (Không được khuyến nghị)
- **READ_COMMITTED** : Ngăn chặn dirty reads. (Mặc định trong nhiều cơ sở dữ liệu như SQL Server, Oracle)
- **REPEATABLE_READ** : Ngăn chặn dirty reads và non-repeatable reads. (Mặc định trong MySQL)
- **SERIALIZABLE** : Ngăn chặn dirty reads, non-repeatable reads và phantom reads. (Mức độ cô lập cao nhất, nhưng có thể ảnh hưởng nghiêm trọng đến hiệu suất)

Cách sử dụng:

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public void processData() {
    // ...
}
```

3.4. Rollback Rules (Quy tắc Rollback)

Mặc định, Spring sẽ rollback một transaction khi một `RuntimeException` hoặc `Error` được ném ra. Nó sẽ không rollback khi một `Checked Exception` được ném ra.

Bạn có thể tùy chỉnh quy tắc rollback bằng cách sử dụng các thuộc tính `rollbackFor`, `noRollbackFor` của `@Transactional`.

- **rollbackFor** : Chỉ định các loại ngoại lệ mà transaction sẽ được rollback khi chúng được ném ra.

```
java @Transactional(rollbackFor = {CustomBusinessException.class,
IOException.class}) public void doSomething() throws IOException { //
... }
```

- **noRollbackFor** : Chỉ định các loại ngoại lệ mà transaction sẽ KHÔNG được rollback khi chúng được ném ra.


```
java @Transactional(noRollbackFor = {UserNotFoundException.class})
public void createUser(User user) { // Nếu UserNotFoundException được
ném ra, transaction vẫn commit }
```

3.5. Timeout và Read-only Transactions

- **timeout**: Đặt thời gian tối đa (tính bằng giây) mà transaction có thể chạy trước khi bị rollback tự động. Giúp ngăn chặn các transaction chạy quá lâu và gây tắc nghẽn tài nguyên.

```
java @Transactional(timeout = 30) // Transaction sẽ rollback nếu chạy
quá 30 giây public void longRunningOperation() { // ... }
```

- **readOnly**: Đánh dấu một transaction là chỉ đọc. Điều này có thể là một tối ưu hóa hiệu suất cho các hoạt động chỉ đọc dữ liệu, vì cơ sở dữ liệu có thể bỏ qua việc ghi log hoặc khóa cần thiết cho các thao tác ghi.

```
java @Transactional(readOnly = true) public List<Product>
getAllProducts() { return productRepository.findAll(); }
```

4. Transaction Managers

`PlatformTransactionManager` là interface trung tâm trong Spring cho việc quản lý transaction. Spring cung cấp nhiều implementation khác nhau của interface này để làm việc với các công nghệ truy cập dữ liệu khác nhau.

4.1. PlatformTransactionManager

Đây là interface cấp cao nhất để quản lý transaction trong Spring. Nó định nghĩa các phương thức cơ bản để lấy, commit và rollback transaction.

4.2. DataSourceTransactionManager

Được sử dụng cho các ứng dụng sử dụng JDBC hoặc iBatis/MyBatis. Nó quản lý transaction trên một `DataSource` duy nhất.

```
@Bean
public PlatformTransactionManager transactionManager(dataSource) {
    return new DataSourceTransactionManager(dataSource);
}
```

4.3. JpaTransactionManager

Được sử dụng cho các ứng dụng sử dụng JPA (Java Persistence API). Nó quản lý transaction thông qua `EntityManagerFactory`.

```
@Bean
public PlatformTransactionManager transactionManager(EntityManagerFactory
entityManagerFactory) {
    return new JpaTransactionManager(entityManagerFactory);
}
```

4.4. HibernateTransactionManager

Được sử dụng cho các ứng dụng sử dụng Hibernate trực tiếp (không thông qua JPA). Nó quản lý transaction thông qua `SessionFactory` của Hibernate.

```
@Bean
public PlatformTransactionManager transactionManager(SessionFactory
sessionFactory) {
    return new HibernateTransactionManager(sessionFactory);
}
```

5. Tích hợp Transaction Management với Spring Boot

Spring Boot tự động cấu hình Transaction Management khi bạn thêm các dependency liên quan, giúp bạn bắt đầu nhanh chóng mà không cần cấu hình phức tạp.

5.1. Auto-configuration cho Transaction Management

Khi bạn thêm `spring-boot-starter-data-jpa` (hoặc các starter khác liên quan đến cơ sở dữ liệu) vào dự án, Spring Boot sẽ tự động cấu hình:

- Một `DataSource` (nếu bạn đã cấu hình thông tin kết nối database).
- Một `PlatformTransactionManager` phù hợp (ví dụ: `JpaTransactionManager` cho JPA, `DataSourceTransactionManager` cho JDBC).

- Kích hoạt `@EnableTransactionManagement` để cho phép sử dụng `@Transactional`.

Điều này có nghĩa là trong hầu hết các trường hợp, bạn chỉ cần thêm `@Transactional` vào các phương thức service của mình và Spring Boot sẽ lo phần còn lại.

5.2. Cấu hình Transaction Manager tùy chỉnh

Nếu bạn cần tùy chỉnh `PlatformTransactionManager` (ví dụ: sử dụng nhiều data source, hoặc các trường hợp phức tạp khác), bạn có thể định nghĩa bean `PlatformTransactionManager` của riêng mình. Spring Boot sẽ tự động sử dụng bean mà bạn định nghĩa thay vì tạo một cái mặc định.

```
package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.PlatformTransactionManager;

import javax.sql.DataSource;

@Configuration
public class CustomTransactionManagerConfig {

    @Bean
    public PlatformTransactionManager customTransactionManager(DataSource
dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }
}
```

Nếu bạn có nhiều `PlatformTransactionManager` bean, bạn cần chỉ định rõ ràng transaction manager nào sẽ được sử dụng cho một `@Transactional` cụ thể bằng cách sử dụng thuộc tính `transactionManager`:

```
@Transactional(transactionManager = "customTransactionManager")
public void someTransactionalMethod() {
    // ...
}
```

6. Best Practices và Lời khuyên

6.1. Sử dụng `@Transactional` đúng cách

- **Đặt `@Transactional` ở tầng Service:** Đây là nơi lý tưởng để đặt `@Transactional` vì nó chứa logic nghiệp vụ và là nơi các thao tác liên quan đến database thường được nhóm lại.
- **Tránh đặt `@Transactional` ở tầng Controller:** Controller chỉ nên xử lý request và gọi các phương thức service. Việc đặt `@Transactional` ở controller có thể dẫn đến các transaction kéo dài không cần thiết.
- **Tránh đặt `@Transactional` ở tầng Repository:** Các phương thức repository thường là các thao tác database đơn lẻ. Transaction nên bao gồm nhiều thao tác liên kết logic với nhau.
- **Chỉ áp dụng cho các phương thức public:** `@Transactional` chỉ hoạt động trên các phương thức public. Nếu bạn đặt nó trên phương thức private hoặc protected, nó sẽ không có tác dụng.
- **Cẩn thận với self-invocation:** Nếu một phương thức `@Transactional` được gọi từ một phương thức khác trong cùng một lớp, transaction sẽ không được áp dụng vì Spring AOP proxy không được sử dụng. Để khắc phục, bạn có thể inject bean của chính lớp đó vào hoặc sử dụng `AopContext.currentProxy()`.

6.2. Xử lý ngoại lệ trong Transaction

- **Chỉ `RuntimeException` và `Error` mới gây rollback mặc định:** Hãy nhớ rằng chỉ các `RuntimeException` và `Error` mới tự động gây rollback. Nếu bạn muốn rollback cho `Checked Exception`, bạn phải chỉ định nó bằng `rollbackFor`.
- **Không bắt ngoại lệ và nuốt chúng:** Nếu bạn bắt một ngoại lệ trong một phương thức `@Transactional` và không ném lại nó (hoặc một `RuntimeException` khác), transaction sẽ không được rollback. Hãy luôn ném lại ngoại lệ hoặc đánh dấu transaction để rollback thủ công
(`TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();`).

6.3. Tránh các vấn đề về hiệu suất

- **Giữ transaction ngắn:** Transaction càng ngắn càng tốt để giảm thiểu việc khóa tài nguyên database và cải thiện hiệu suất đồng thời.

- **Sử dụng `readOnly = true` cho các thao tác chỉ đọc:** Điều này có thể giúp tối ưu hóa hiệu suất cho các truy vấn đọc.
- **Cẩn thận với `LazyInitializationException` :** Nếu bạn truy cập các mối quan hệ lazy-loaded bên ngoài transaction, bạn sẽ gặp lỗi này. Đảm bảo rằng tất cả các dữ liệu cần thiết đã được tải trong phạm vi transaction.

6.4. Kiểm thử Transaction

Luôn kiểm thử các kịch bản transaction để đảm bảo rằng dữ liệu được commit hoặc rollback đúng cách. Spring cung cấp `@Transactional` cho các test method, cho phép test chạy trong một transaction và tự động rollback sau khi test hoàn thành.

```

package com.example.demo.service;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.transaction.annotation.Transactional;

import static org.junit.jupiter.api.Assertions.assertThrows;

@SpringBootTest
public class ProductServiceIntegrationTest {

    @Autowired
    private ProductService productService;
    @Autowired
    private ProductRepository productRepository;

    @Test
    @Transactional // Mô phỏng test chạy trong một transaction và rollback sau đó
    void whenCreateProductAndErrorOccurs_thenRollback() {
        Product product = new Product("Test Product", 10.0, 5);
        // Giả lập lỗi trong service để kiểm tra rollback
        // product.setName("error"); // Giả sử service ném lỗi nếu tên là
"error"

        assertThrows(RuntimeException.class, () -> {
            productService.createProduct(product);
        });

        // Kiểm tra xem sản phẩm có được lưu vào DB không (nếu rollback thành
công thì không)
        // Optional<Product> foundProduct = productRepository.findByName("Test
Product");
        // assertFalse(foundProduct.isPresent());
    }

    @Test
    @Transactional
    void whenUpdateStock_thenCommit() {
        Product product = new Product("Existing Product", 20.0, 10);
        productRepository.save(product);

        productService.updateProductStock(product.getId(), 5);

        Product updatedProduct =
productRepository.findById(product.getId()).orElseThrow();
        // assertEquals(15, updatedProduct.getStock());
    }
}

```

Kết luận

Spring Security và Transaction Management là hai trụ cột quan trọng trong việc phát triển các ứng dụng Spring Boot mạnh mẽ và đáng tin cậy. Bằng cách hiểu rõ các khái niệm, kiến trúc và cách sử dụng các tính năng mà Spring cung cấp, bạn có thể xây dựng các ứng dụng an toàn, có khả năng mở rộng và duy trì tính toàn vẹn dữ liệu một cách hiệu quả. Luôn

tuân thủ các best practices và kiểm thử kỹ lưỡng để đảm bảo chất lượng ứng dụng của bạn.

Tài liệu tham khảo

[1] Spring Security Official Documentation: <https://docs.spring.io/spring-security/reference/index.html> [2] Baeldung - Spring Security: <https://www.baeldung.com/spring-security> [3] Spring Framework Official Documentation - Transaction Management: <https://docs.spring.io/spring-framework/reference/data-access/transaction.html> [4] Baeldung - Spring Transaction Management: <https://www.baeldung.com/spring-transaction-configuration> [5] GeeksforGeeks - Spring Boot Transaction Management: <https://www.geeksforgeeks.org/spring-boot-transaction-management-using-transactional-annotation/>